

Chapter 6: Backend API Core

Welcome back to the CinéConnect tutorial! In [Chapter 5: Real-time Communication](#), we explored how [Socket.io](#) enables instant, two-way conversations, bringing live chat and dynamic updates to our app. Now, let's zoom out and look at the "brain" of CinéConnect: the **Backend API Core**.

The Brain of CinéConnect: Why We Need a Backend API

Imagine CinéConnect as a bustling city. The beautiful user interface you see and interact with (the frontend) is like all the buildings, roads, and parks. But for that city to actually *function*, it needs a central control system: a power grid, a water supply, a traffic control center, and systems to manage its citizens' records. This is precisely what our **Backend API Core** does for CinéConnect.

The backend is where all the critical server-side logic lives. It's responsible for:

- **Managing user accounts:** Storing your profile, handling login/registration ([Chapter 2: User Authentication](#)).
- **Storing movie data:** Keeping track of which movies users favorite, rate, or add to watchlists.
- **Handling interactions:** Processing your requests to like a movie, follow another user, or send a chat message.
- **Communicating with external services:** Like fetching movie details from [TMDB](#) ([Chapter 4: Third-Party Movie Data \(TMDB\)](#)).

Without a robust backend, our beautiful frontend would just be an empty shell. It needs a reliable partner to store data, perform complex operations, and ensure everything works securely.

Our main goal in this chapter is to understand the core technologies and principles that power CinéConnect's backend, allowing it to efficiently handle all server-side logic and data interactions.

Let's use a common action as our example: **A user wants to "like" a movie.** This involves the frontend telling the backend to record this preference, and the backend confirming it.

The Core Technologies: [Express.js](#), [TypeScript](#), [Drizzle ORM](#), [PostgreSQL](#)

Our backend is built using a powerful combination of tools, each playing a vital role:

1. [Express.js](#) - The City's Traffic Controller:

- This is a popular framework for Node.js that helps us build web applications and APIs.
- Think of [Express.js](#) as the traffic controller that directs incoming requests (like "someone wants to like movie X") to the right handler in our backend. It listens for specific "routes" (like street addresses) and ensures the correct function processes the request.

2. [TypeScript](#) - The Detailed Blueprint:

- This is a language that adds "types" (like defining what kind of data should go where) to JavaScript.

- Using **TypeScript** is like having a detailed blueprint for our city. It helps us write structured, reliable, and bug-free code by catching errors *before* the code even runs, making development smoother and safer.

3. **PostgreSQL - The City's Archives (Database):**

- This is a powerful and reliable database system.
- **PostgreSQL** is where all of CinéConnect's important information is stored: user accounts, movie ratings, favorite lists, chat messages, and so on. It's our city's meticulously organized archive where data is kept safe and can be quickly retrieved.

4. **Drizzle ORM - The Specialized Translator:**

- **ORM** stands for "Object-Relational Mapper." **Drizzle ORM** is a modern tool that helps our backend code talk to the **PostgreSQL** database.
- Instead of writing complex SQL commands (the specific language databases understand), **Drizzle ORM** acts like a specialized translator. You write code in **TypeScript** (JavaScript with types), and **Drizzle** automatically translates it into the correct SQL commands for **PostgreSQL**, and then translates the database's response back into **TypeScript** objects. This makes database interactions much easier and less prone to errors.

How CinéConnect Handles "Liking a Movie"

Let's trace what happens when you click the "Like" button on a movie in CinéConnect.

The User Experience

1. You're on a movie details page in CinéConnect.
2. You see a "Like" button (perhaps a heart icon).
3. You click the button.
4. The button instantly changes to show it's "liked" (e.g., a filled heart), and perhaps a small confirmation appears.

Using the Backend API Core

From the frontend's perspective, this interaction is typically a simple API call to our backend.

```
// apps/frontend/src/service/movies.service.ts (Conceptual)
import { apiClient } from "../lib/api-client"; // Client to talk to OUR backend

export const moviesService = {
  // ... other methods like getTrending ...

  async likeMovie(movieId: number): Promise<boolean> {
    const response = await apiClient.post( // Send a POST request
      `/api/v1/movies/${movieId}/like`,    // To this specific backend endpoint
      {}                                   // No body needed for a simple like
    );
    return response.success; // Assuming backend sends { success: true }
  },
};
```

```
async unlikeMovie(movieId: number): Promise<boolean> {
  const response = await apiClient.delete( // Send a DELETE request
    `/api/v1/movies/${movieId}/like`      // To the same endpoint
  );
  return response.success;
},
};
```

When you click "Like" in the frontend, the `moviesService.likeMovie(movieId)` function is called.

- **Input:** `movieId` (e.g., `550` for "Fight Club").
- **Output:** A `Promise<boolean>` which resolves to `true` if the like was successful, `false` otherwise.

This function makes an HTTP `POST` request to our backend's `/api/v1/movies/:movieId/like` endpoint. The `POST` method indicates that we are *creating* a "like" record. If you were to "unlike" a movie, it would typically be a `DELETE` request to the same endpoint.

Under the Hood: The Backend Processing a "Like"

Now, let's peel back the layers and see how our backend processes this "like" request. [Backend Processing](#)

Here's a step-by-step breakdown:

1. **Frontend Sends Request:** The `Frontend` sends a `POST` request to `/api/v1/movies/550/like`.
2. **Express Receives:** Our `Express App` receives this incoming request. It acts as the "traffic controller" and looks at the URL and HTTP method.
3. **Route Matching:** `Express` matches this request to a specific route handler in our `Movie Controller`.
4. **Movie Controller:** The `Movie Controller` is responsible for handling the incoming request. It extracts information like the `movieId` (550) and the logged-in `userId` (from the authentication middleware, as seen in [Chapter 2: User Authentication](#)). It then delegates the actual work to the `Movie Service`.
5. **Movie Service:** The `Movie Service` contains the business logic. It knows *how* to record a "like." It tells `Drizzle ORM` to create a new `favorite` entry.
6. **Drizzle ORM Translates:** `Drizzle ORM` takes the `TypeScript` instruction from the `Movie Service` and translates it into a standard SQL `INSERT` command for `PostgreSQL`.
7. **PostgreSQL Stores Data:** The `PostgreSQL DB` executes the SQL command, saving the new favorite record.
8. **Response Flow:** The confirmation flows back from `PostgreSQL` through `Drizzle ORM`, `Movie Service`, `Movie Controller`, and `Express App` until the `Frontend` receives a successful response.

Diving into the Code

Let's look at key simplified snippets that make this happen.

1. Defining Our Database Schema (`db/schema.ts`)

Before we can store likes, we need to tell `Drizzle ORM` what our `favorites` table looks like in `PostgreSQL`.

```
// apps/backend/src/db/schema.ts (Simplified)
import { pgTable, serial, integer, timestamp, unique } from "drizzle-orm/pg-core";
import { users } from "./schema"; // Reference to our users table

export const favorites = pgTable("favorites", {
  id: serial("id").primaryKey(), // Auto-incrementing ID
  userId: integer("user_id") // Who liked the movie
    .references(() => users.id, { onDelete: 'cascade' }), // Links to user ID
  externalMovieId: integer("external_movie_id").notNull(), // The TMDB movie ID
  addedAt: timestamp("added_at").defaultNow(), // When it was liked
}, (t) => ({
  // Ensures a user can only like a movie once
  pk: unique().on(t.userId, t.externalMovieId),
}));
// ... other tables like users, comments, etc.
```

Here, we define a `favorites` table with columns for a unique `id`, the `userId` (linking to the `users` table), `externalMovieId` (the ID from TMDB), and `addedAt` timestamp. The `unique().on(t.userId, t.externalMovieId)` constraint ensures a user can only favorite a specific movie once.

2. Express Setup and Routing (`app.ts`)

`app.ts` is where our main `Express` application is configured and all the different routes are set up.

```
// apps/backend/src/app.ts (Simplified)
import express, { Express } from 'express';
// ... other middleware imports like cors, cookieParser, helmet ...
import movieRoutes from './routes/movie.route'; // Our movie-specific routes

const app: Express = express();
// ... (apply CORS, security headers, cookie parser, etc.) ...

app.use(express.json()); // Essential: allows Express to parse JSON request bodies

const API_V1 = '/api/v1';
// This tells Express: "For any request starting with /api/v1/movies, use movieRoutes"
app.use(`${API_V1}/movies`, movieRoutes);

// ... (other route registrations, error handlers) ...

export default app;
```

- `app.use(express.json())`: This middleware is crucial! It tells `Express` to automatically parse incoming request bodies if they are in JSON format.
- `app.use(`${API_V1}/movies`, movieRoutes)`: This line registers our `movieRoutes`. It means that any request to `/api/v1/movies/...` will be handled by the logic defined in `movieRoutes`.

3. Defining Movie Routes (`routes/movie.route.ts`)

This file defines the specific URL paths and HTTP methods for movie-related actions.

```
// apps/backend/src/routes/movie.route.ts (Simplified)
import { Router } from 'express';
import { MovieController } from '../controllers/movie.controller';
import { authMiddleware } from '../middlewares/auth.middleware'; // For protected routes

const router = Router();

// Route to get trending movies (from Chapter 4)
router.get('/trending', MovieController.getTrending);

// Route to like a movie: requires authentication (authMiddleware)
router.post('/:movieId(\\d+)/like', authMiddleware, MovieController.likeMovie);
// Route to unlike a movie: requires authentication
router.delete('/:movieId(\\d+)/like', authMiddleware,
MovieController.unlikeMovie);

// ... other movie routes ...

export default router;
```

- `router.post('/:movieId(\\d+)/like', authMiddleware, MovieController.likeMovie)`: This line sets up our "like movie" route.
 - `router.post`: This means it handles HTTP **POST** requests.
 - `/:movieId(\\d+)/like`: This is the URL path. `movieId` is a parameter, and `(\\d+)` ensures it's a number. So, `/123/like` matches, but `/abc/like` does not.
 - `authMiddleware`: This is our security checkpoint from [Chapter 2: User Authentication](#). It ensures that only logged-in users can "like" a movie. If the user is not authenticated, this middleware will block the request.
 - `MovieController.likeMovie`: If authentication passes, this is the function that will be executed to handle the request.

4. Handling the Request in the Controller (`controllers/movie.controller.ts`)

The controller is the first point of contact for a specific request logic. It orchestrates the flow.

```
// apps/backend/src/controllers/movie.controller.ts (Simplified)
import { Request, Response } from 'express';
import { success, badRequest } from '../utils'; // Helper for sending responses
import { MovieService } from '../services/movie.service'; // Our business logic service

export const MovieController = {
  // ... getTrending (from Chapter 4) ...
```

```

    async likeMovie(req: Request, res: Response) {
      // req.user is set by authMiddleware from Chapter 2
      const userId = req.user?.userId;
      const movieId = parseInt(req.params.movieId, 10); // Get movieId from URL

      if (!userId) { // Should not happen with authMiddleware, but good practice
        return badRequest(res, 'Authentication required');
      }
      if (isNaN(movieId)) {
        return badRequest(res, 'Invalid movie ID');
      }

      await MovieService.addFavorite(userId, movieId); // Delegate to service
      return success(res, { message: 'Movie liked' }, 201); // Send success
    },

    async unlikeMovie(req: Request, res: Response) {
      const userId = req.user?.userId;
      const movieId = parseInt(req.params.movieId, 10);

      if (!userId || isNaN(movieId)) {
        return badRequest(res, 'Invalid request');
      }

      await MovieService.removeFavorite(userId, movieId);
      return success(res, { message: 'Movie unliked' });
    },
  };

```

- `req.user?.userId`: The `authMiddleware` (from [Chapter 2: User Authentication](#)) attaches the `userId` to the request object, making it available here.
- `req.params.movieId`: This extracts the `movieId` (e.g., `550`) from the URL.
- `MovieService.addFavorite(userId, movieId)`: The controller delegates the actual database interaction to the `MovieService`. This keeps controllers light and focused on request handling.
- `success(res, { message: 'Movie liked' }, 201)`: If the service call is successful, the controller sends an HTTP 201 Created status back to the frontend.

5. Business Logic and Database Interaction in the Service (`services/movie.service.ts`)

The service layer contains the core business logic and interacts with the database. This is where `Drizzle ORM` shines.

```

// apps/backend/src/services/movie.service.ts (Simplified)
import { db } from '../db'; // Our Drizzle ORM database client
import { favorites } from '../db/schema'; // Our favorites table schema
import { eq, and } from 'drizzle-orm'; // Drizzle ORM query helpers

export const MovieService = {
  // ... getTrending (from Chapter 4, which delegates to TmdbService) ...

```

```
    async addFavorite(userId: number, externalMovieId: number) {
      // Drizzle ORM: Insert a new row into the favorites table
      await db.insert(favorites).values({ userId, externalMovieId })
        .onConflictDoNothing({
          target: [favorites.userId, favorites.externalMovieId]
        }); // If already exists, do nothing
    },

    async removeFavorite(userId: number, externalMovieId: number) {
      // Drizzle ORM: Delete a row from the favorites table
      await db.delete(favorites)
        .where(and(
          eq(favorites.userId, userId),
          eq(favorites.externalMovieId, externalMovieId)
        ));
    },

    async isMovieFavoritedByUser(userId: number, externalMovieId: number):
    Promise<boolean> {
      // Drizzle ORM: Check if a movie is favorited by a user
      const favorite = await db.query.favorites.findFirst({
        where: and(
          eq(favorites.userId, userId),
          eq(favorites.externalMovieId, externalMovieId)
        ),
      });
      return !!favorite; // Return true if found, false otherwise
    }
  };
};
```

- **db**: This is our **Drizzle ORM** client, configured to connect to **PostgreSQL**. It's how we interact with the database.
- **favorites**: We import our **favorites** schema, which **Drizzle** uses to understand the table structure.
- **db.insert(favorites).values(...)**: This is **Drizzle ORM** in action! Instead of writing raw SQL **INSERT** statements, we use **TypeScript** objects. **Drizzle** translates this into the correct SQL.
- **.onConflictDoNothing(...)**: This **Drizzle** feature elegantly handles cases where a user tries to "like" a movie they've already liked (thanks to our **unique** constraint in **db/schema.ts**).
- **db.delete(favorites).where(...)**: Another **Drizzle** example for deleting records. **eq** and **and** are **Drizzle** functions for building SQL **WHERE** clauses easily in **TypeScript**.

This structured approach, from **Express** routing to **Controller** handling, **Service** business logic, and **Drizzle ORM** database interaction, is the backbone of our Backend API Core.

Summary of Backend API Core Components

Here's a quick overview of how the core backend components work together:

Component	Role	Key Technologies	Example Action (Like Movie)
-----------	------	------------------	-----------------------------

Component	Role	Key Technologies	Example Action (Like Movie)
Express.js	Directs incoming HTTP requests to the right handler.	Node.js, Express	Receives <code>POST /api/v1/movies/:id/like</code> and routes it to <code>MovieController</code> .
TypeScript	Adds type safety and structure to JavaScript code.	TypeScript	Ensures <code>movieId</code> is a number and <code>userId</code> is present, catches errors at development time.
Controller	Handles incoming requests, extracts data, delegates to services.	Express, TypeScript	Parses <code>movieId</code> from URL, gets <code>userId</code> from request, calls <code>MovieService.addFavorite</code> .
Service	Contains business logic, interacts with the database.	TypeScript, Drizzle ORM	Calls <code>Drizzle ORM</code> to insert a record into the <code>favorites</code> table for the specific user/movie.
Drizzle ORM	Translates TypeScript code to SQL and vice-versa.	Drizzle ORM	Translates <code>db.insert(favorites).values(...)</code> into SQL: <code>INSERT INTO favorites (...) VALUES (...)</code> .
PostgreSQL	Stores all application data securely and reliably.	PostgreSQL	Stores the new favorite record in the <code>favorites</code> table.

Conclusion

In this chapter, we've explored the foundational "brain" of CinéConnect: its Backend API Core. We learned how `Express.js` acts as the traffic controller, directing requests, how `TypeScript` provides a reliable blueprint for structured code, and how `PostgreSQL` serves as our secure data archive. Most importantly, we saw how `Drizzle ORM` acts as a specialized translator, allowing our `TypeScript` services to easily communicate with `PostgreSQL` without needing to write raw SQL. This powerful combination ensures that CinéConnect can efficiently manage all server-side logic, user data, and interactions.

Now that we understand how our backend processes requests, in the next chapter, we'll dive into how CinéConnect ensures that all incoming API requests are valid and secure using [API Request Validation](#).